

Módulo 2

Do Repositório Local ao GitHub

Versão expandida para PDF

Material linearizado a partir do curso HTML interativo. Inclui conteúdos, descrições de imagem, atividades, feedbacks, gabaritos formativos, quiz e glossário.

Sumário

1. Página 1: Capa
2. Página 2: Autoria
3. Página 3: Boas-vindas!
4. Página 4: Objetivos de aprendizagem
5. Página 5: Introdução
6. Página 6: 1. Do ambiente local ao ecossistema distribuído
7. Página 7: 2. Comandos essenciais de criação e inspeção
8. Página 8: 3. Registro de alterações: add, commit e commits atômicos
9. Página 9: 4. Higiene do repositório e uso de .gitignore
10. Página 10: 5. GitHub e repositórios remotos
11. Página 11: 6. Sincronização local-remota: remote, push, fetch e pull
12. Página 12: 7. Situações comuns de rejeição de push e diagnóstico
13. Página 13: 8. Recuperação de erros: princípios gerais
14. Página 14: 9. Recuperação de erros na prática: reset, restore e revert
15. Página 15: 10. Padronização das mensagens com Conventional Commits
16. Página 16: 11. Boas práticas e ética no versionamento
17. Página 17: 12. Documentação básica do projeto: README, histórico, licença e referências
18. Página 18: Conclusão
19. Página 19: Referências
20. Página 20: Quiz final
21. Glossário

1. Capa

Trilha: Git e GitHub

Módulo 2 - Do Repositório Local ao GitHub



Fluxo de sincronização entre um repositório local e o GitHub, destacando versionamento, envio de alterações, colaboração, revisão e segurança do projeto.

Imagem gerada por IA

Menu lateral Acompanhe capítulos e páginas visitadas.	Navegação Use Próximo, Anterior ou os dots.
Práticas Complete os desafios para consolidar comandos.	Simuladores Teste efeitos de push, fetch, pull, reset e revert.
Glossário Pesqu	Quiz

2. Autoria

Autoria

Prof. Dr. Eduardo Amadeu Moresi

Prof. Ms. Weldes Lima

Equipe Técnica

Eduardo Amadeu Dutra Moresi

Vilson Hartmann

Mário Braga

Everson Menezes

Isolda Gusmão

Thiago Ribalovsky

Weldes Lima Oliveira

Projeto

Residência em TIC 04B - TIC EM TRILHAS

Direitos Autorais

© 2026 – TIC em Trilhas - Universidade Católica de Brasília UCB

Todos os direitos reservados.

É proibida a reprodução total ou parcial deste material, por qualquer meio, sem autorização expressa da executora.

Sobre o TIC em trilhas

Iniciativa em parceria com TIC em Trilhas e parte integrante do projeto Residência em TIC 04B. Projetos apoiados pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248/91, sendo uma iniciativa do Ministério da Ciência, Tecnologia e Inovações e Softex no âmbito do Programa MCTI Futuro.



Régua institucional com as marcas TIC em Trilhas, Universidade Católica de Brasília, Softex, MCTI Futuro, Ministério da Ciência, Tecnologia e Inovação e Governo Federal.

3. Boas-vindas!

Boas-vindas!

Neste módulo o foco é deslocar a sua aprendizagem da compreensão conceitual para a prática tecnicamente orientada do ciclo local-remoto: você vai criar, clonar, inspecionar, versionar, publicar, sincronizar e corrigir projetos com apoio do Git e do GitHub.

Cada capítulo trás conceitos, comandos, justificativas, riscos, boas práticas, exemplos e propostas de exercícios. A intenção não é transformá-lo em mero executor de comandos, mas em usuário capaz de interpretar estados do repositório, compreender os efeitos de cada operação, escolher estratégias de recuperação de erros e produzir evidências auditáveis de aprendizagem em um repositório publicado no GitHub.

Missão prática do módulo

Marque as ações que você deverá realizar em um repositório real ao estudar este módulo.

- ☐ Criar ou clonar um repositório. ☐ Registrar commits claros. ☐ Publicar no GitHub. ☐ Simular fetch, pull e push. ☐ Documentar uma recuperação de erro.

4. Objetivos de aprendizagem

Objetivos de aprendizagem

Ao concluir este módulo, você será capaz de:

Inicializar, clonar e configurar repositórios Git, criando ambientes locais e remotos adequados para o controle de versão de projetos.

Inspecionar o estado e o histórico de alterações em um repositório, utilizando comandos como git status, git diff e git log para acompanhar mudanças nos arquivos.

Registrar e sincronizar alterações no Git e no GitHub, realizando commits claros, envio, busca e atualização de arquivos entre repositórios locais e remotos.

Resolver erros e organizar o fluxo de trabalho com branches, remotes e mensagens padronizadas, aplicando boas práticas de recuperação, mesclagem e escrita de commits.

Diagnóstico inicial

Antes de estudar, indique seu nível de segurança em cada ação.

Usar git status 2 Publicar no GitHub 2 Resolver erro de push 1

5. Introdução

Introdução

Neste módulo, você vai compreender a relação complementar entre Git e GitHub e aprender a transitar com segurança entre o ambiente local e o ambiente remoto. O Git será apresentado como um sistema de controle de versão distribuído, responsável por registrar o histórico de alterações de um projeto, enquanto o GitHub será abordado como uma plataforma de hospedagem e colaboração, usada para publicar repositórios, compartilhar mudanças, revisar contribuições e organizar o trabalho em equipe. Ao longo do módulo, você vai aprender a criar e configurar repositórios, registrar commits, sincronizar alterações, lidar com remotes, resolver erros comuns e adotar boas práticas de organização e documentação. O objetivo é que, ao final, você seja capaz de publicar um repositório funcional no GitHub, com README, .gitignore, histórico de commits padronizados, simulação de sincronização e registro documentado de recuperação de erro, demonstrando autonomia prática e compreensão conceitual do fluxo de versionamento.

6. 1. Do ambiente local ao ecossistema distribuído

1. Do ambiente local ao ecossistema distribuído

O primeiro eixo da prática local-remoto é compreender que seu trabalho ocorre simultaneamente em dois espaços. O primeiro é o ambiente local: a máquina na qual você edita arquivos, executa comandos e cria commits. O segundo é o ambiente remoto: o repositório hospedado no GitHub, acessível via navegador, protocolos de rede e comandos Git. A passagem entre esses espaços não é automática; ela é mediada por operações explícitas de publicação e sincronização.

O repositório local é completo. Em um sistema distribuído, você não possui apenas a última versão dos arquivos, mas uma cópia do histórico do projeto. Essa característica diferencia Git de modelos puramente centralizados e permite trabalho offline, inspeção de histórico sem conexão, commits locais e experimentação controlada. Mesmo quando o GitHub está temporariamente indisponível, você pode continuar registrando commits localmente e sincronizar posteriormente.

O repositório remoto, no contexto deste módulo, é criado no GitHub. Ele funciona como ponto de compartilhamento, backup, visibilidade e coordenação. Em turmas de graduação, o GitHub também pode servir como portfólio, espaço de entrega, instrumento de revisão docente e ambiente de colaboração entre colegas. A literatura sobre uso educacional do GitHub destaca que a plataforma pode aproximar estudantes de práticas autênticas de engenharia de software, mas também exige orientação cuidadosa sobre privacidade, curvas de aprendizagem e interpretação de métricas de contribuição.

É fundamental que você diferencie Git de GitHub. Git é uma ferramenta de controle de versão instalada localmente. GitHub é um serviço online que hospeda repositórios Git e adiciona recursos sociais e colaborativos — issues, pull requests, páginas de projeto, discussões, ações automatizadas e gestão de permissões. Confundir ambos prejudica a aprendizagem: quando um comando falha no terminal, o problema pode estar no Git local, na autenticação, no remote, na rede ou em permissões do GitHub. A distinção ajuda no diagnóstico.

O conceito de remote torna a relação local-remota operacional. Um remote é um nome associado a uma URL de repositório. Por convenção, o remote principal recebe o nome origin. Quando você executa `git push origin main`, está dizendo ao Git para enviar commits da branch main para o repositório remoto chamado origin. Quando executa `git fetch origin`, solicita que o Git busque atualizações desse repositório remoto.

No GitHub, a criação de repositórios envolve decisões de visibilidade. Um repositório público pode ser visto por qualquer pessoa; um repositório privado restringe acesso conforme permissões. Para turmas de graduação, a escolha depende da política da instituição, da natureza dos dados e da finalidade pedagógica. Projetos contendo dados sensíveis, informações pessoais ou materiais avaliativos devem exigir cautela. Mesmo em repositórios públicos, `.gitignore` e boas práticas de documentação são necessários para evitar exposição indevida.

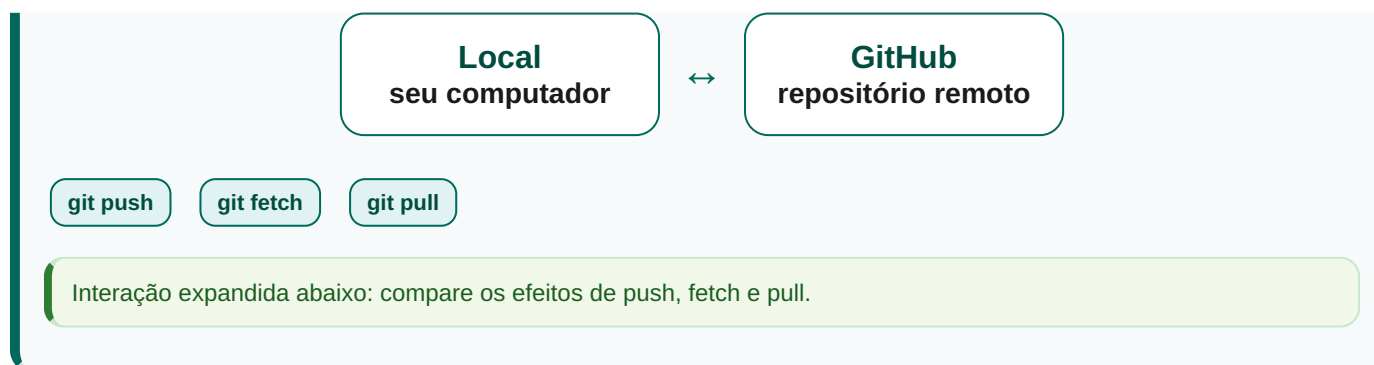
Compreenda que a publicação no GitHub não substitui sua responsabilidade local. Um repositório remoto não corrige automaticamente commits mal organizados, mensagens obscuras ou arquivos sensíveis. Ao contrário, publicar amplia o alcance das suas decisões locais. Um commit ruim localmente pode ser corrigido com `reset`; um commit ruim publicado exige maior cautela, geralmente `revert`. Portanto, a passagem para o ecossistema distribuído exige maturidade: o histórico deixa de ser privado e passa a afetar outras pessoas.

Essa compreensão sustenta o restante do módulo. Quando você aprender `git init`, `git clone`, `status`, `diff`, `log`, `add`, `commit`, `remote`, `push`, `fetch`, `pull`, `restore`, `reset`, `revert` e Conventional Commits, cada comando será interpretado como parte de um sistema sociotécnico. Git registra estados; GitHub compartilha estados; pessoas interpretam estados. A competência profissional nasce da articulação entre esses três níveis.

Simulador local-remoto

Clique nos comandos para ver para onde a informação se move.

PDF expandido - Módulo 2: Do Repositório Local ao GitHub



Decisão rápida

Git ou GitHub?

Registra snapshots locais

Cria pull requests na web

Configura remote origin

Interações expandidas / gabarito formativo

Simulador push/fetch/pull

- git push: envia commits locais para o GitHub.
- git fetch: baixa referências remotas sem alterar os arquivos locais.
- git pull: busca e integra alterações remotas ao trabalho local.

Classificação Git ou GitHub

- Registra snapshots locais - correto: Git é a ferramenta local de controle de versão.
- Cria pull requests na web - atenção: GitHub hospeda repositórios Git e recursos colaborativos.
- Configura remote origin - correto: origin é uma configuração do Git que aponta para uma URL remota.

7. 2. Comandos essenciais de criação e inspeção

2. Comandos essenciais de criação e inspeção

Os comandos essenciais de criação e inspeção constituem a alfabetização operacional do Git. Antes de publicar no GitHub, você precisa criar ou obter um repositório, verificar o estado dos arquivos, comparar alterações e ler o histórico. Esses comandos são `git init`, `git clone`, `git status`, `git diff`, `git log` e `git log --oneline`. Eles não devem ser encarados como lista de instruções independentes, mas como instrumentos de investigação do estado de um projeto versionado.

`git init` prepara uma pasta para controle de versão. Tecnicamente, o comando cria o diretório `.git`, que armazenará objetos, referências, configurações locais e metadados do repositório. Didaticamente, `git init` demarca o início de uma história rastreável. Antes dele, a pasta é apenas um conjunto de arquivos; depois dele, ela passa a ter memória versionada. Execute `git init` em uma pasta vazia e observe que os arquivos de trabalho permanecem visíveis, mas surge uma infraestrutura oculta.

`git clone` é usado quando o projeto já existe em outro repositório. Ao clonar, o Git copia arquivos, histórico e configuração do remote de origem. Diferentemente de baixar um arquivo ZIP do GitHub, o clone preserva a natureza versionada do projeto. Essa distinção é central: baixar ZIP permite ver arquivos, mas não permite sincronizar histórico com facilidade. Clonar cria uma relação entre o repositório local e o remoto, normalmente configurando origin automaticamente.

`git status` é o seu principal comando de orientação. Ele informa a branch atual, a existência de alterações preparadas para commit, alterações não preparadas e arquivos não rastreados. Em termos pedagógicos, status é o mapa do repositório. Execute `git status` antes de `git add`, antes de `git commit`, antes de `git push` e depois de operações de recuperação. Essa prática reduz erros e desenvolve uma postura investigativa.

`git diff` permite observar alterações linha a linha. Enquanto status indica que algo mudou, diff mostra o conteúdo da mudança. Isso ajuda a evitar commits acidentais, especialmente quando você edita vários arquivos. O comando também favorece revisão individual antes da publicação. Uma atividade simples consiste em modificar uma linha no README, executar `git diff` e descrever o significado dos sinais de adição e remoção exibidos.

`git log` exibe o histórico de commits. Ele mostra identificadores, autores, datas e mensagens. A versão `git log --oneline` reduz a saída a uma visualização compacta, adequada para acompanhar a sequência de alterações. A leitura do log deve ser associada à qualidade das mensagens. Um histórico composto por mensagens como "ajuste", "teste" e "coisa nova" é pouco informativo; um histórico com "docs: atualiza README" e "fix: corrige validação" comunica evolução.

A inspeção também se relaciona à aprendizagem metacognitiva. Ao observar status, diff e log, você aprende a formular perguntas sobre seu próprio trabalho: o que alterei? o que está preparado? o que será commitado? qual foi a última mudança? essa modificação pertence ao mesmo commit ou deveria ser separada? Essas perguntas são mais importantes do que a memorização mecânica de comandos, pois orientam decisões em situações novas.

No laboratório, crie uma pasta chamada `modulo2-local-remoto`, execute `git init`, crie `README.md`, observe `git status`, adicione conteúdo, execute `git diff`, prepare o arquivo com `git add`, faça commit e consulte `git log --oneline`. Em seguida, clone um repositório próprio do GitHub em outra pasta, compare a experiência de init e clone e registre as diferenças no relatório.

```
mkdir modulo2-local-remoto cd modulo2-local-remoto git init echo "# Projeto de prática GitHub" > README.md git status git diff git add README.md git commit -m "docs: adiciona README inicial" git log --oneline
```

Console guiado: criação e inspeção

Clique na sequência correta para criar um repositório, verificar estado e registrar o README.

Sequência sugerida: mkdir modulo2-local-remoto → git init → git status → git add README.md → git commit -m "docs: adiciona README inicial"

mkdir modulo2-local-remoto

git init

git status

git add README.md

git commit

Conferir sequência

Feedback: sequência adequada - criar pasta, iniciar Git, inspecionar, preparar e commitar.

Interações expandidas / gabarito formativo

Sequência recomendada da prática

- mkdir modulo2-local-remoto
- git init
- git status
- git add README.md
- git commit -m "docs: adiciona README inicial"
- Feedback: sequência adequada - criar pasta, iniciar Git, inspecionar, preparar e commitar.

8. 3. Registro de alterações: add, commit e commits atômicos

3. Registro de alterações: add, commit e commits atômicos

Registrar alterações em Git é um ato técnico e comunicativo. O comando `git add` seleciona o conteúdo que integrará o próximo snapshot; o comando `git commit` consolida esse conteúdo no histórico. A separação entre preparar e confirmar é uma das características mais importantes do Git, pois permite que você organize mudanças de modo granular. Nem toda alteração realizada no diretório de trabalho precisa necessariamente entrar no próximo commit.

`git add` não significa simplesmente adicionar arquivo ao Git uma vez para sempre. O comando prepara uma versão específica do arquivo para o próximo commit. Se você executa `git add` em `codigo.js` e depois modifica `codigo.js` novamente antes do commit, o Git passa a enxergar duas versões relevantes: a versão preparada e a versão modificada posteriormente. O índice guarda um estado; o diretório de trabalho pode avançar para outro estado antes do commit.

`git commit` cria o registro histórico. Um commit deve representar uma unidade lógica de mudança. Essa unidade pode ser uma correção de erro, uma pequena documentação, uma configuração inicial ou uma funcionalidade delimitada. Commits atômicos são commits que agrupam apenas alterações relacionadas. Eles facilitam revisão, reversão, busca de bugs e compreensão do projeto. Em contextos acadêmicos, também facilitam a avaliação da sua participação, pois mostram uma progressão clara de trabalho.

A prática de commits atômicos contrasta com dois padrões problemáticos. O primeiro é o commit gigante, no qual muitas alterações independentes são registradas de uma só vez. O segundo é o commit excessivamente fragmentado, no qual cada pequena edição sem significado lógico vira um commit separado. O equilíbrio formativo consiste em registrar mudanças pequenas, porém coerentes.

A mensagem de commit é parte do registro. Ela deve descrever a alteração de maneira compreensível para alguém que lerá o histórico no futuro. Um commit com mensagem "docs: adiciona instruções de execução" informa tipo e finalidade. Um commit com mensagem "atualiza coisas" não permite avaliar o que mudou nem por quê.

Separe alterações de correção, documentação e funcionalidade. Se você corrige um erro em `script.js` e, ao mesmo tempo, melhora o README, faça dois commits: "fix: corrige cálculo de média" e "docs: atualiza instruções de uso". Essa separação favorece entendimento e permite reverter a correção sem remover a documentação, se necessário. Insista nessa disciplina desde o início, antes mesmo de introduzir branches.

A relação entre add e commit também ensina responsabilidade. O Git não decide sozinho o que deve entrar no histórico: você seleciona. Isso significa que arquivos temporários, credenciais ou resultados gerados podem ser evitados antes do commit. A etapa de preparação é momento de revisão. Execute `git status` e `git diff` antes de `git add`; depois de `git add`, execute `git status` novamente para verificar o que está staged; e somente então realize `git commit`.

O laboratório pode solicitar que você crie três mudanças simultâneas: alterar README, criar `script.js` e criar `anotacoes.txt`. Em seguida, prepare apenas README e `script.js` para um commit de documentação e estrutura, deixando `anotacoes.txt` fora. Depois, decida se `anotacoes.txt` será versionado, ignorado ou removido. Essa prática demonstra que o Git oferece controle seletivo e que o histórico deve ser construído com intencionalidade.

Simulador add → commit

Selecione quais arquivos entram em um commit atômico de documentação e estrutura.

☐ README.md

☐ script.js

☐ anotacoes.txt

☐ debug.log

Validar commit

Feedback: escolha README.md e script.js para um commit atômico coerente; evite debug.log e anotações soltas.

Interações expandidas / gabarito formativo

Validação de commit atômico

- Selecionar: README.md e script.js.
- Evitar: debug.log e anotacoes.txt.
- Feedback positivo: README e script.js formam uma unidade coerente. Anotações e logs devem ficar fora ou ser avaliados separadamente.

9. 4. Higiene do repositório e uso de .gitignore

4. Higiene do repositório e uso de .gitignore

A higiene do repositório diz respeito à seleção criteriosa do que deve ou não ser versionado. Nem todo arquivo presente em uma pasta de projeto pertence ao histórico Git. Alguns arquivos são produtos derivados, como executáveis, pastas de build, arquivos temporários e caches. Outros são dependências externas que podem ser reconstruídas por gerenciadores de pacote. Outros ainda contêm informações sensíveis, como senhas, tokens, chaves privadas, arquivos .env e credenciais de serviços. Versionar esses arquivos pode gerar problemas técnicos, legais e éticos.

O arquivo .gitignore é o mecanismo mais comum para declarar padrões de arquivos que o Git deve ignorar quando ainda não estão rastreados. Ele pode listar nomes específicos, extensões, diretórios e padrões globais. Por exemplo, node_modules/ ignora a pasta de dependências de um projeto Node.js; *.log ignora arquivos de log; .env ignora arquivos de variáveis de ambiente; dist/ e build/ ignoram produtos de compilação. Atenção: a existência de .gitignore não remove automaticamente arquivos já rastreados, por isso o momento de criação é importante.

A finalidade do .gitignore é múltipla. Em primeiro lugar, reduz ruído em git status, permitindo que você veja apenas alterações relevantes. Em segundo lugar, diminui o tamanho do repositório ao evitar arquivos gerados ou pesados. Em terceiro lugar, previne conflitos desnecessários em arquivos que variam entre ambientes locais. Em quarto lugar, protege dados sensíveis — essa última função deve ser enfatizada, pois plataformas como GitHub são indexadas e monitoradas; segredos publicados podem ser capturados rapidamente por agentes automatizados.

O caso de arquivos sensíveis merece tratamento formal. Um arquivo .env pode conter senha de banco de dados, token de API ou chave secreta. Uma chave privada SSH jamais deve ser publicada. Versionamento é memória persistente: mesmo que um segredo seja removido em commit posterior, ele pode continuar no histórico se já foi commitado anteriormente.

Quando um arquivo sensível foi criado mas ainda não foi commitado, a solução é simples: adicionar padrão ao .gitignore e não executar git add. Quando o arquivo já foi commitado localmente, pode ser necessário remover do índice com git rm --cached e fazer novo commit. Quando foi enviado ao GitHub, a medida imediata é revogar a credencial e substituí-la. Essa distinção introduz pensamento de resposta a incidentes.

Crie um .gitignore funcional como parte do produto de aprendizagem. O arquivo deve conter padrões adequados ao projeto escolhido. Mesmo que o projeto seja simples, inclua exemplos pedagógicos e explique no README por que esses padrões foram selecionados. Essa explicação reforça que .gitignore não é cópia de modelo sem reflexão, mas decisão contextual de higiene.

Exemplo inicial de .gitignore para o módulo *.log .env senha.txt node_modules/ dist/ build/ .DS_Store

Monte seu .gitignore

Escolha os arquivos que devem ser ignorados.

☐ debug.log

☐ .env

☐ node_modules/

☐ README.md

☐ S|

PDF expandido - Módulo 2: Do Repositório Local ao GitHub

Gerar .gitignore

```
*.log  
.env  
node_modules/  
build/
```

Feedback: padrões adequados para higiene e segurança do repositório.

Interações expandidas / gabarito formativo

Geração de .gitignore

- Padrões adequados: *.log, .env, node_modules/ e build/.
- Cuidado: README.md e src/app.js normalmente pertencem ao histórico e não devem ser ignorados.
- Feedback: padrões adequados favorecem higiene e segurança do repositório.

10. 5. GitHub e repositórios remotos

5. GitHub e repositórios remotos

A publicação de um projeto no GitHub é uma etapa formativa de grande valor, pois transforma o trabalho local em artefato compartilhável. Entretanto, publicar não significa apenas clicar em um botão. É necessário compreender o papel do repositório remoto, a visibilidade do projeto, a configuração do remote origin, a autenticação e a relação entre branch local e branch remota. Sem essas bases, você pode repetir comandos copiados sem entender por que funcionam ou falham.

A criação de conta no GitHub deve ser acompanhada por orientações de identidade digital. O nome de usuário pode se tornar parte do seu portfólio. O e-mail associado pode aparecer em commits, salvo configurações de privacidade. É adequado explicar que o GitHub pode funcionar como portfólio técnico, mas que você deve tomar decisões conscientes sobre exposição de dados pessoais e trabalhos acadêmicos.

Ao criar um repositório, você precisa escolher nome, descrição, visibilidade, licença e inicialização com README. Há dois fluxos comuns. No primeiro, você cria o repositório vazio no GitHub e depois associa o projeto local existente com `git remote add origin <URL>`. No segundo, cria o repositório no GitHub com README e o clona localmente com `git clone`. Ambos são válidos, mas têm consequências. Inicializar README no GitHub e também criar commits locais antes de sincronizar pode gerar históricos divergentes se você não compreender pull.

O nome origin é uma convenção. Em projetos simples, há um único remote chamado origin. O comando `git remote -v` mostra URLs de fetch e push associadas ao remote. Se o URL estiver errado, push e fetch falharão. Se o remote não existir, será necessário adicioná-lo. Se o repositório foi clonado, origin provavelmente já estará configurado.

A autenticação no GitHub evoluiu ao longo do tempo. Operações via HTTPS podem exigir token ou integração com gerenciadores de credenciais; operações via SSH exigem par de chaves configurado. Embora o módulo não aprofunde criptografia ou segurança de acesso, fique atento: sua senha de conta não deve ser colocada em scripts, README ou arquivos de configuração. A autenticação é fronteira entre prática local-remota e segurança operacional.

A associação entre projeto local e repositório remoto deve ser demonstrada passo a passo. Você cria o repositório local, faz ao menos um commit, cria repositório vazio no GitHub, copia a URL, executa `git remote add origin`, confere com `git remote -v` e envia com `git push -u origin main`. A opção `-u` configura o upstream, permitindo que futuros `git push` e `git pull` saibam qual branch remota acompanhar.

A interface web do GitHub complementa a linha de comando. Verifique se os arquivos apareceram, abra o histórico de commits, observe a mensagem e confira o README renderizado. Essa etapa fecha o ciclo: o commit criado localmente se torna visível remotamente. A visibilidade do histórico reforça a importância de boas mensagens, .gitignore e documentação.

Gerador de comando remote

Preencha a URL do repositório remoto e gere o comando de associação.

Gerar comandoCopiar

```
git remote add origin https://github.com/usuario/projeto.git
git remote -v
git push -u origin main
```

Depois use: `git remote -v` e `git push -u origin main`

Interações expandidas / gabarito formativo

Comando gerado para remote

- `git remote add origin https://github.com/usuario/projeto.git`
- `git remote -v`
- `git push -u origin main`

11. 6. Sincronização local-remota: remote, push, fetch e pull

6. Sincronização local-remota: remote, push, fetch e pull

A sincronização local-remota é um dos pontos que mais geram confusão para iniciantes. A causa principal é a tendência de pensar no GitHub como pasta em nuvem sincronizada automaticamente. Git e GitHub não operam assim. Suas alterações locais não aparecem no GitHub até que sejam commitadas e enviadas por push. Alterações feitas no GitHub não aparecem na sua máquina local até que sejam buscadas por fetch ou integradas por pull. Essa mediação explícita é uma força, não uma deficiência: ela permite controle sobre o que será compartilhado.

`git remote` é o comando de administração dos repositórios remotos. Com `git remote -v`, você visualiza os remotes configurados. Com `git remote add origin <URL>`, adiciona um remote. Com `git remote remove origin`, remove a associação. Com `git remote set-url origin <URL>`, altera a URL. Essas operações devem ser utilizadas com cuidado, pois uma URL incorreta pode enviar dados para repositório errado ou impedir comunicação.

`git push` envia commits locais para o repositório remoto. O comando não envia alterações não commitadas. Se você modifica um arquivo, mas não executa `git add` e `git commit`, `git push` não terá o que enviar. Publicar no GitHub exige uma cadeia: editar, inspecionar, preparar, commitar, enviar.

`git fetch` baixa informações do remote sem alterar o diretório de trabalho. Essa operação atualiza referências remotas, como `origin/main`, permitindo comparar o histórico local com o remoto. Por ser não invasiva, `fetch` é excelente para inspeção. Execute `git fetch origin` e depois `git log --oneline --graph --decorate --all` para observar diferenças.

`git pull` é uma operação composta. Em configuração padrão, `pull` executa `fetch` e depois integra alterações remotas à branch local, geralmente por `merge`. Isso significa que `pull` pode modificar arquivos de trabalho e gerar conflitos. Execute `git status` antes de `pull` e evite `pull` com alterações locais não commitadas, a menos que você saiba exatamente o que está fazendo.

A diferença entre baixar histórico e integrar alterações pode ser demonstrada com uma simulação. Publique um README no GitHub. Depois, edite o README diretamente pela interface web, simulando alteração de colega. Localmente, execute `git fetch origin` e observe que o arquivo local ainda não mudou. Em seguida, execute `git pull origin main` e observe que a alteração aparece no diretório de trabalho. Essa experiência concreta fixa a distinção conceitual.

A rejeição de push é uma situação didática importante. Quando o remote possui commits que a branch local não possui, o Git rejeita o push para proteger o histórico remoto. A mensagem pode assustar, mas revela uma regra de segurança: antes de publicar, integre o que já foi publicado por outros. Leia a mensagem, execute `fetch`, analise diferenças e depois `pull`. Essa sequência forma uma atitude profissional diante de conflitos de sincronização.

A sincronização tem dimensão colaborativa. O hábito de `fetch` frequente reduz surpresas; o hábito de `pull` antes de iniciar nova tarefa minimiza divergências; o hábito de push ao final de uma unidade de trabalho compartilha progresso. Em contextos acadêmicos, esses hábitos ajudam a evitar entregas de última hora com conflitos não resolvidos.

Comando	O que faz	Altera arquivos locais?	Uso recomendado
<code>git remote -v</code>	Lista remotes e URLs	Não	Conferir associação com GitHub
<code>git push</code>	Envia commits locais	Não altera arquivos locais	Publicar trabalho após commit
<code>git fetch</code>	Baixa referências e objetos	Não altera diretório de trabalho	Inspecionar mudanças remotas com

git pull	Baixa e integra alterações remotas	Sim, pode alterar arquivos e gerar conflito	Atualizar trabalho local após inspeção
----------	------------------------------------	---	--

Fetch, pull ou push?

Escolha o comando adequado para cada situação.

Situação 1: Quero ver se há mudanças no GitHub sem alterar meus arquivos locais.

/ push / fetch / pull

Situação 2: Tenho commits locais prontos e quero publicá-los no GitHub.

/ push / fetch / pull

Situação 3: Quero trazer e integrar no meu trabalho uma alteração feita no GitHub.

/ push / fetch / pull

12. 7. Situações comuns de rejeição de push e diagnóstico

7. Situações comuns de rejeição de push e diagnóstico

A rejeição de push é uma das primeiras experiências de fricção entre iniciantes e GitHub. Ela ocorre quando o Git identifica que atualizar a branch remota com os commits locais não seria um avanço simples e seguro — geralmente porque o repositório remoto contém commits que a sua máquina ainda não possui. Em vez de sobrescrever o remoto, o Git bloqueia a operação. Interprete essa recusa como mecanismo de proteção do histórico compartilhado.

Uma situação comum ocorre quando você cria um repositório no GitHub já inicializado com README e também cria um repositório local com commit inicial independente. Ao tentar push, há dois históricos que não compartilham a mesma origem. Você precisa integrar o histórico remoto, compreender a divergência e decidir o procedimento adequado. Para evitar essa situação no primeiro fluxo, crie repositório remoto vazio ou clone o repositório inicializado pelo GitHub.

Outra situação comum ocorre quando você edita um arquivo diretamente na interface do GitHub e depois continua trabalhando localmente sem buscar a alteração remota. Ao tentar push, a operação é rejeitada porque o remoto avançou. Essa simulação é pedagogicamente útil, pois representa o trabalho de um colega. Você aprende que GitHub não sincroniza automaticamente e que o trabalho remoto precisa ser incorporado por fetch e pull antes do envio local.

O diagnóstico começa por `git status` e `git remote -v`. O primeiro informa a situação da branch local; o segundo confirma o destino. Depois, `git fetch origin` atualiza informações remotas. Em seguida, `git log --oneline --graph --decorate --all` ajuda a visualizar divergências. A visualização do grafo reduz a sensação de magia.

A solução mais simples para rejeição de push em cenário controlado é executar `git pull origin main`, resolver eventuais conflitos e tentar push novamente. Entretanto, pull pode gerar merge commit ou conflito. Em atividades iniciais, faça a alteração remota em linha diferente da alteração local para evitar conflito no primeiro contato.

Em alguns casos, você pode encontrar mensagens sobre branch main ou master. Isso ocorre porque repositórios antigos usavam master como nome padrão. Verifique o nome da branch com `git branch` e adapte os comandos. Se a branch local é main, use main. Se é master, use master, ou renomeie conforme política da trilha.

⚠️ Atenção: `git push --force` pode sobrescrever histórico remoto e prejudicar colaboradores. No Módulo 2, evite esse comando. A solução pedagógica é integrar mudanças, não apagar trabalho remoto.

Ao final da seção, registre uma rejeição de push simulada. O relatório deve conter: descrição do cenário, mensagem resumida do erro, diagnóstico realizado, comando de integração usado e resultado final. Essa documentação transforma o erro em evidência de aprendizagem.

Diagnóstico de push rejeitado

Você editou o README no GitHub e também fez um commit local. O push foi rejeitado. Monte a sequência segura.

Sequência: `git status` → `git fetch origin` → `git pull origin main` → `git push`

`git status`

`git fetch origin`

`git pull origin main`

`git push`

`git push --force`

Conferir diagnóstico

Feedback: evite push --force neste módulo. Primeiro diagnostique e integre o histórico remoto.

Interações expandidas / gabarito formativo

Diagnóstico seguro para push rejeitado

- git status
- git fetch origin
- git pull origin main
- git push
- Evitar neste módulo: git push --force.
- Feedback: verificar estado, buscar remoto, integrar e só então publicar.

13. 8. Recuperação de erros: princípios gerais

8. Recuperação de erros: princípios gerais

A recuperação de erros é um dos temas mais relevantes para a sua confiança em Git. Muitos iniciantes temem perder trabalho, quebrar o repositório ou tornar o projeto irrecuperável. Esse medo é compreensível, mas pode ser reduzido por um princípio: antes de escolher um comando de recuperação, identifique em que estágio o erro ocorreu. O erro aconteceu antes de git add? Depois de git add, mas antes de commit? Depois de commit local? Depois de push? Cada estágio exige estratégia distinta.

Erros antes do commit geralmente são os mais simples. Se um arquivo foi modificado de modo indesejado e a modificação ainda não foi registrada, git restore <arquivo> pode restaurar o estado do último commit. Use com cautela, pois descarta mudanças locais. Se você apenas preparou um arquivo por engano, git restore --staged <arquivo> remove o arquivo da staging area sem apagar o conteúdo do diretório de trabalho.

Erros depois de commit local, mas antes de push, permitem maior flexibilidade. Como o histórico ainda não foi compartilhado, você pode usar git reset para mover a referência da branch e reorganizar commits. git reset --soft HEAD~1 desfaz o último commit mantendo alterações no índice. git reset --mixed HEAD~1 desfaz o commit e retira alterações do índice, preservando arquivos modificados. git reset --hard HEAD~1 desfaz commit e descarta alterações, sendo destrutivo. Hard exige confirmação consciente.

Erros depois de envio ao GitHub exigem cautela colaborativa. Se um commit já foi publicado, outros podem tê-lo clonado ou usado como base. Reescrever histórico compartilhado pode causar divergências e confusão. Nesses casos, git revert é a estratégia segura: cria um novo commit que desfaz os efeitos de um commit anterior, preservando a sequência histórica. Nem todo erro deve ser apagado; em contextos colaborativos, muitas vezes é melhor registrar a correção de forma transparente.

A diferença entre reset e revert é conceitual. Reset move referências e pode reescrever a percepção do histórico local. Revert adiciona história. Reset é apropriado para reorganização local antes de compartilhar. Revert é apropriado para desfazer alterações já compartilhadas. Perguntar "o erro já foi compartilhado? preciso preservar histórico? desejo manter alterações nos arquivos?" é mais produtivo que simplesmente perguntar "qual comando devo usar?"

A recuperação de erros também envolve segurança com arquivos sensíveis. Se senha.txt foi criado mas não commitado, basta ignorar e remover. Se foi commitado localmente, remova do histórico local antes do push. Se foi enviado ao GitHub, a senha deve ser considerada comprometida — revogar credencial é prioridade.

Situação	Comando recomendado	Efeito principal	Risco
Alteração local não desejada antes do commit	git restore <arquivo>	Restaura arquivo ao último commit	Perde mudanças locais não salvas
Arquivo preparado por engano	git restore --staged <arquivo>	Remove da staging area	Não remove conteúdo do arquivo
Commit local ainda não publicado	git reset --soft HEAD~1	Desfaz commit mantendo alterações staged	Reescreve histórico local
Commit local com alterações a reorganizar	git reset --mixed HEAD~1	Desfaz commit e limpa staging	Exige novo git add
Commit local a descartar completamente	git reset --hard HEAD~1	Remove commit e alterações	Destrutivo
Commit já publicado	git revert <hash>	Cria commit inverso	Mantém histórico, pode exigir revisão

Árvore de decisão: restore, reset ou revert

Escolha o comando mais seguro para cada estágio do erro.

Alteração local indesejada antes do commit

Arquivo preparado por engano

Commit local ainda não publicado

Commit já enviado ao GitHub

Resumo: restore para alteração local; restore --staged para staging area; reset para commit local não publicado; revert para commit publicado.

Interações expandidas / gabarito formativo

Escolha de recuperação por situação

- Alteração local indesejada antes do commit: use `git restore <arquivo>`.
- Arquivo preparado por engano: use `git restore --staged <arquivo>` para remover da staging area sem apagar o arquivo.
- Commit local ainda não publicado: use `reset` para reorganizar commit local ainda não publicado.
- Commit já enviado ao GitHub: use `revert` para desfazer commit já publicado preservando o histórico.

14. 9. Recuperação de erros na prática: reset, restore e revert

9. Recuperação de erros na prática: reset, restore e revert

A prática de recuperação deve ser conduzida em ambiente controlado. Você precisa experimentar comandos de desfazer sem medo, mas com consciência dos efeitos. Para isso, realize a atividade em repositório de laboratório, com arquivos simples e commits identificáveis. A sequência começa com `git status` limpo, cria uma alteração, registra a alteração, desfaz e observa o resultado.

No primeiro exercício, crie ou modifique um arquivo `script.js` introduzindo uma linha que representa erro. Antes de preparar o arquivo, execute `git diff` para ver a mudança. Em seguida, execute `git restore script.js` e confira que a alteração desapareceu. Pergunta reflexiva: por que o Git conseguiu restaurar o arquivo? A resposta deve remeter ao último commit como referência conhecida.

No segundo exercício, modifique `README.md`, execute `git add README.md` e perceba que não queria incluir essa alteração no próximo commit. Use `git restore --staged README.md`. Depois, execute `git status` e observe que o arquivo permanece modificado, mas não está mais staged. Esse exercício reforça a diferença entre índice e diretório de trabalho.

No terceiro exercício, crie um commit local com mensagem inadequada, por exemplo "teste". Em seguida, execute `git reset --soft HEAD~1`. O commit desaparece do log, mas as alterações permanecem preparadas. Crie novo commit com mensagem padronizada, como "docs: melhora instruções de execução". Essa atividade demonstra que `reset --soft` é útil para corrigir commit local antes do compartilhamento.

No quarto exercício, use `git reset --mixed HEAD~1`. Observe que as alterações continuam nos arquivos, mas deixam de estar preparadas. Perguntas reflexivas: o que mudou no histórico? o que mudou na staging area? o que permaneceu no diretório de trabalho? A resposta evidencia as três árvores.

⚠ Use `git reset --hard` apenas em arquivo descartável e com plena consciência. Iniciantes não devem depender do reflog para recuperar trabalho perdido.

No quinto exercício, simule um erro publicado: crie um commit, envie ao GitHub, identifique o hash e execute `git revert <hash>`. O Git criará um commit de reversão. Acesse o GitHub e observe que o commit original e o commit de reversão permanecem no histórico. Essa visualização é essencial para entender preservação de rastreabilidade.

Ao final, produza um documento `recuperacao-de-erros.md` com uma matriz de decisões contendo: estágio do erro, comando escolhido, justificativa, resultado e risco evitado.

```
# Exemplo de correção de commit local com reset --soft echo "linha com erro" >> script.js
git add script.js git commit -m "teste"
git log --oneline git reset --soft HEAD~1 git status # editar o arquivo ou apenas corrigir a mensagem
git commit -m "fix: corrige comportamento do script"
```

Laboratório guiado de recuperação

1. Criar alteração com erro em `script.js`
2. Executar `git diff` antes de preparar
3. Testar `git restore script.js`
4. Criar commit local com mensagem ruim

5.

Aplicar git reset --soft HEAD~1

6.

Refazer commit com Conventional Commit

Interações expandidas / gabarito formativo

Roteiro da prática de recuperação

- 1. Criar alteração com erro em script.js.
- 2. Executar git diff antes de preparar.
- 3. Testar git restore script.js.
- 4. Criar commit local com mensagem ruim.
- 5. Aplicar git reset --soft HEAD~1.
- 6. Refazer commit com Conventional Commit.

15. 10. Padronização das mensagens com Conventional Commits

10. Padronização das mensagens com Conventional Commits

A padronização de mensagens de commit é uma prática de comunicação técnica. O histórico de um projeto não é apenas sequência de estados; é narrativa de decisões. Quando mensagens são inconsistentes, vagas ou excessivamente informais, o histórico perde valor como documento de engenharia. Conventional Commits propõe uma estrutura simples, legível por humanos e máquinas: <tipo>[escopo opcional]: <descrição>.

O tipo informa a natureza da mudança. feat indica nova funcionalidade; fix indica correção; docs indica alteração de documentação; style indica mudanças de formatação que não afetam lógica; refactor indica reestruturação sem alterar comportamento externo; test indica criação ou modificação de testes; chore indica tarefas auxiliares; perf indica melhoria de desempenho.

O escopo opcional delimita área do projeto. Em "docs(readme): atualiza instruções de instalação", o escopo readme indica onde a mudança ocorreu. Para projetos pequenos, o escopo pode ser omitido.

A descrição deve ser curta, específica e orientada à mudança. Boas descrições usam verbo no presente ou forma direta: adiciona, corrige, atualiza, remove, refatora. Exemplos adequados: "docs: adiciona seção de execução", "fix: corrige nome do arquivo no README" e "chore: adiciona regras de gitignore". Exemplos inadequados: "alterei umas coisas", "update", "trabalho final".

No contexto educacional, Conventional Commits auxilia avaliação. Um professor que examina o histórico consegue identificar documentação, correções e melhorias. O próprio estudante, ao retornar semanas depois, reconstrói seu raciocínio. Essa rastreabilidade é coerente com a finalidade do controle de versão como infraestrutura de memória e colaboração.

A atividade prática deve exigir ao menos cinco commits padronizados. Por exemplo: "docs: adiciona README inicial"; "chore: adiciona .gitignore"; "feat: cria script de exemplo"; "fix: corrige saída do script"; "docs: documenta recuperação de erro". Em seguida, crie CHANGELOG.md manualmente, agrupando mudanças por tipo.

Tipo	Uso recomendado	Exemplo em português
feat	Nova funcionalidade ou comportamento observável	feat: adiciona script de saudacao
fix	Correção de erro	fix: corrige nome do arquivo no README
docs	Alterações de documentação	docs: descreve comandos de execução
style	Formatação sem impacto lógico	style: ajusta indentacao do script
refactor	Reestruturação sem nova funcionalidade	refactor: reorganiza funcoes auxiliares
test	Testes ou exemplos verificáveis	test: adiciona caso de teste simples
chore	Manutenção, configuração ou tarefas auxiliares	chore: adiciona regras de gitignore
perf	Melhoria de desempenho	perf: reduz repeticao em processamento

Gerador de Conventional Commit

Tipo

docs / feat / fix / chore / refactor / test / style / perf

Escopo opcional

readme

Descrição

adiciona README inicial

```
docs(readme): adiciona README inicial
```

Validar mensagem

Feedback: mensagem válida e informativa.

Interações expandidas / gabarito formativo

Mensagem de Conventional Commit

- Exemplo gerado: docs(readme): adiciona README inicial.
- Formato esperado: tipo(escopo opcional): descrição objetiva.
- Tipos aceitos: feat, fix, docs, style, refactor, test, chore e perf.

16. 11. Boas práticas e ética no versionamento

11. Boas práticas e ética no versionamento

O Git não é apenas uma ferramenta técnica; é também infraestrutura de colaboração e responsabilidade. Cada commit registra autoria, data, conteúdo e mensagem. Quando publicado no GitHub, esse registro pode ser lido, revisado, reutilizado e auditado por outras pessoas. Por isso, boas práticas de versionamento têm dimensão ética. Elas protegem colegas, professores, usuários, instituições e você mesmo contra ambiguidade, perda de dados e exposição indevida.

A primeira boa prática é não commitar diretamente na branch principal em projetos colaborativos. A branch main deve representar uma linha estável do projeto. No Módulo 2, você pode trabalhar diretamente em main por simplicidade, mas saiba que essa é uma simplificação didática, não modelo completo de equipe.

A segunda prática é commitar cedo e frequentemente. Isso não significa registrar qualquer tecla pressionada, mas criar pontos de recuperação em intervalos lógicos. Em trabalhos acadêmicos, esperar até o último dia para realizar um commit gigante impede rastreabilidade e dificulta feedback.

A terceira prática é explicar o motivo da mudança. O código mostra o que mudou; a mensagem deve ajudar a compreender por que a mudança foi necessária. "fix: corrige divisao por zero" é melhor que "fix: corrige erro", pois indica a causa técnica.

A quarta prática é não manter código morto comentado. O Git já preserva versões. Manter código morto polui arquivos, confunde leitores e dificulta manutenção. Quando uma implementação não é mais necessária, deve ser removida e registrada em commit.

A quinta prática é proteger dados sensíveis. Nenhuma senha, token, chave privada ou credencial deve entrar no repositório. Repositórios públicos podem ser indexados por buscadores e acessados por terceiros.

A sexta prática é respeitar autoria. Commits devem ser realizados com identidade correta. Em trabalhos em grupo, um estudante não deve commitar trabalho de outro como se fosse próprio sem acordo explícito. Pesquisas educacionais alertam que métricas do GitHub podem ser enganosas se interpretadas sem contexto.

A sétima prática é documentar decisões. README, CHANGELOG e relatórios curtos em Markdown ajudam a contextualizar o histórico. Um repositório com commits bem escritos, mas sem instruções de execução, ainda é incompleto.

Casos éticos: o que fazer?

Caso: você percebeu que adicionou `.env` antes de commitar.

Ignorar com `.gitignore` antes do commit

Fazer push e apagar depois

Caso: você está em projeto colaborativo e quer desfazer commit publicado.

Usar `git revert`

Usar `git push --force`

Interações expandidas / gabarito formativo

Feedback das decisões éticas

- Ignorar com `.gitignore` antes do commit - correto: inclua `.env` no `.gitignore` e não faça `git add`.
- Fazer push e apagar depois - risco: o push publicaria a credencial no GitHub.
- Usar `git revert` - correto: `revert` preserva o histórico compartilhado.
- Usar `git push --force` - evite neste módulo; ele pode sobrescrever histórico remoto.

17. 12. Documentação básica do projeto: README, histórico, licença e referências

12. Documentação básica do projeto: README, histórico, licença e referências

A documentação é parte integrante do repositório, não adorno posterior. O README é geralmente o primeiro arquivo lido por quem acessa um projeto no GitHub. Ele orienta interpretação, execução e avaliação. Em formação de graduação, escrever README desenvolve sua capacidade de comunicação técnica, competência tão relevante quanto escrever código.

A estrutura mínima de README para este módulo deve conter título, descrição, objetivos, tecnologias utilizadas, requisitos, modo de execução, estrutura de pastas, exemplos de uso, histórico resumido, autores, licença e referências.

A seção de histórico pode ser manual no Módulo 2, por meio de CHANGELOG.md simples. Consulte `git log --oneline` e traduza as mudanças em uma lista organizada. Esse exercício mostra que commits padronizados facilitam documentação.

A licença define condições de uso, cópia, modificação e distribuição. Em projetos acadêmicos simples, pode-se optar por uma licença permissiva, como MIT. Publicar no GitHub não equivale automaticamente a permitir qualquer uso. A ausência de licença pode gerar incerteza jurídica.

As referências são essenciais quando o projeto se apoia em materiais externos. Em trabalhos de graduação, registrar referências previne plágio e demonstra rigor acadêmico.

A documentação também pode incluir relatório de aprendizagem. Crie um arquivo `relatorio-modulo2.md` com três seções: diferenças entre Git e GitHub; explicação de fetch, pull e push; relato de recuperação de erro.

O README deve ser escrito em Markdown. Visualize-o na interface web após push, verificando se a formatação está correta. Essa validação fecha o ciclo entre texto local e publicação remota.

Modelo mínimo de README.md # Projeto de Prática GitHub ## Descrição Breve explicação do objetivo do projeto. ## Como executar 1. Clone o repositório. 2. Abra a pasta do projeto. 3. Execute o script conforme orientação. ## Histórico de aprendizagem - Uso de git init, status, add e commit. - Publicação no GitHub com git push. - Sincronização com fetch e pull. ## Autoria Nome do estudante. ## Licença Definir conforme orientação da disciplina.

Montador de README

Preencha as seções mínimas e copie o rascunho para seu repositório.

```
# Projeto

## Descrição

## Como executar
1. Clone o repositório.
2. Abra a pasta do projeto.
3. Siga as instruções do módulo.

## Histórico de aprendizagem
- Us
- Pu
```

PDF expandido - Módulo 2: Do Repositório Local ao GitHub

- Sincronização com fetch e pull.
- Registro de recuperação de erro.

Autoria

Nome do estudante

Licença

Definir conforme orientação da disciplina.

Interações expandidas / gabarito formativo

Modelo de README gerado

- # Projeto
- ## Descrição
- ## Como executar
- 1. Clone o repositório.
- 2. Abra a pasta do projeto.
- 3. Siga as instruções do módulo.
- ## Histórico de aprendizagem
- - Uso de git init, status, add e commit.
- - Publicação no GitHub com git push.
- - Sincronização com fetch e pull.
- - Registro de recuperação de erro.
- ## Autoria
- Nome do estudante
- ## Licença
- Definir conforme orientação da disciplina.

18. Conclusão

Conclusão

Até aqui você deve ter superado a etapa de uso local isolado do Git e ingressado no ciclo local-remoto com GitHub. Você será capaz de criar e clonar repositórios, inspecionar alterações, registrar commits, configurar remotes, publicar no GitHub, sincronizar mudanças, proteger o repositório com .gitignore, recuperar erros e comunicar alterações com mensagens padronizadas.

Um projeto não é apenas conjunto de arquivos; é histórico de decisões. Um commit não é apenas salvamento; é registro de uma unidade lógica de mudança. Um push não é backup automático; é publicação de commits. Um pull não é atualização inocente; é integração de alterações remotas. Um .gitignore não é detalhe; é mecanismo de higiene e segurança. Um README não é formalidade; é interface de comunicação do projeto.

A partir desse ponto, você está preparado para aprender branches, pull requests, merge, conflitos e rebase. O Módulo 3 assumirá que você já sabe construir commits de qualidade e sincronizá-los com GitHub. Um histórico ruim torna revisão mais difícil; mensagens vagas prejudicam colaboração; falta de .gitignore aumenta conflitos; desconhecimento de fetch e pull dificulta atualização de branches.

Você não precisa memorizar todos os comandos, mas deve saber consultar documentação, interpretar mensagens do Git, explicar estados e escolher procedimentos seguros. Essa autonomia é o diferencial entre uso superficial e prática profissional. Git e GitHub tornam-se, assim, instrumentos de aprendizagem, colaboração, documentação e responsabilidade técnica.

Checklist final de evidências

- ☐ Repositório publicado no GitHub. ☐ README com objetivo e instruções. ☐ .gitignore funcional. ☐ Pelo menos cinco commits padronizados. ☐ Simulação de fetch/pull registrada.
- ☐ Recuperação de erro documentada.

19. Referências

Referências

CHACON, Scott; STRAUB, Ben. Pro Git. 2. ed. Tradução para português. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 18 ago. 2024.

CÓDIGO FONTE TV. Code Review no GitHub. YouTube, 2023. Disponível em: https://www.youtube.com/watch?v=2alg7MQ6_sl. Acesso em: 18 ago. 2024.

DUDLER, Roger. Git - Guia prático. 2023. Disponível em: https://rogerdudler.github.io/git-guide/index.pt_BR.html. Acesso em: 18 ago. 2024.

GITHUB. GitHub Docs. Disponível em: <https://docs.github.com/>. Acesso em: 18 ago. 2024.

20. Quiz final

Quiz final

Responda às questões para concluir o módulo. A nota de corte é 75%.

1. O que o git push envia ao GitHub?

1. Arquivos não salvos
2. Commits locais **(correta)**
3. Alterações feitas só no navegador
4. A pasta inteira sem histórico

Feedback: Push envia commits locais já registrados.

2. Qual comando baixa informações remotas sem alterar o diretório de trabalho?

1. git pull
2. git fetch **(correta)**
3. git push
4. git init

Feedback: Fetch é a operação mais segura para inspeção.

3. O que o .gitignore ajuda a evitar?

1. Commits claros
2. Versionamento de arquivos indevidos **(correta)**
3. Criação de README
4. Uso de branches

Feedback: .gitignore reduz ruído e protege arquivos sensíveis ainda não rastreados.

4. Um commit já publicado deve ser desfeito preferencialmente com:

1. git revert **(correta)**
2. git reset --hard
3. apagar a pasta
4. git init

Feedback: Revert preserva o histórico compartilhado.

5. Qual mensagem segue Conventional Commits?

1. alterei coisas
2. docs: adiciona README inicial (**correta**)
3. update
4. trabalho final

Feedback: Ela informa tipo e descrição objetiva.

6. O que git remote -v mostra?

1. Arquivos modificados
2. URLs de remotes configurados (**correta**)
3. Histórico completo
4. Credenciais salvas

Feedback: remote -v mostra os destinos de fetch e push.

7. Por que um push pode ser rejeitado?

1. Porque o remoto tem commits que o local não possui (**correta**)
2. Porque README é obrigatório
3. Porque git status foi usado
4. Porque o GitHub sincroniza automaticamente

Feedback: O Git protege o histórico remoto contra sobrescrita insegura.

8. O que é um commit atômico?

1. Commit com muitas mudanças desconexas
2. Commit pequeno e coerente sobre uma unidade lógica (**correta**)
3. Commit sem mensagem
4. Commit feito apenas no GitHub

Feedback: Commits atômicos facilitam revisão, diagnóstico e reversão.

[Ver resultado](#)

Glossário

Git

Sistema distribuído de controle de versão que registra snapshots do projeto.

GitHub

Plataforma online para hospedar repositórios Git e apoiar colaboração.

Remote

Nome associado à URL de um repositório remoto. O padrão é origin.

Push

Envia commits locais para o repositório remoto.

Fetch

Baixa informações remotas sem alterar o diretório de trabalho.

Pull

Executa fetch e integra alterações remotas na branch local.

Commit

Registro histórico de uma unidade lógica de mudança.

.gitignore

Arquivo que define padrões de arquivos não rastreados que devem ser ignorados.

Reset

Move referências locais e pode reorganizar commits ainda não publicados.

Revert

Cria um novo commit que desfaz outro, preservando o histórico compartilhado.

Conventional Commits

Padrão de mensagens do tipo tipo(escopo): descrição.
